

# Gemini-TTS

Try Gemini-TTS in Vertex AI Studio (<https://console.cloud.google.com/agent-platform/studio/media/speech>)

Try in Colab ([https://colab.research.google.com/github/GoogleCloudPlatform/generative-ai/blob/main/audio/speech/getting-started/gemini\\_3\\_1\\_flash\\_tts.ipynb](https://colab.research.google.com/github/GoogleCloudPlatform/generative-ai/blob/main/audio/speech/getting-started/gemini_3_1_flash_tts.ipynb))

View notebook on GitHub ([https://github.com/GoogleCloudPlatform/generative-ai/blob/main/audio/speech/getting-started/gemini\\_3\\_1\\_flash\\_tts.ipynb](https://github.com/GoogleCloudPlatform/generative-ai/blob/main/audio/speech/getting-started/gemini_3_1_flash_tts.ipynb))

Gemini-TTS is the latest evolution of our Cloud TTS technology that moves beyond natural-sounding speech and provides granular control over generated audio using text-based prompts. Using Gemini-TTS, you can synthesize single or multi-speaker speech from short snippets to long-form narratives, precisely dictating style, accent, pace, tone, and even emotional expression, all steerable through natural-language prompts.

## Available models

Gemini-TTS includes the following available models:

|                                                                                                                                  |                                                                                                                                                                                                                                                                                    |
|----------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">Gemini 3.1 Flash TTS (Preview)</a> <a href="#">Gemini 2.5 Flash TTS...</a> More ▾<br>(#gemini-3.1-flash-tts-preview) |                                                                                                                                                                                                                                                                                    |
| Model ID                                                                                                                         | gemini-3.1-flash-tts-preview                                                                                                                                                                                                                                                       |
| Optimized for                                                                                                                    | Low latency, controllable, single- and multi-speaker Cloud TTS audio generation for cost-efficient everyday applications                                                                                                                                                           |
| Input and output modalities                                                                                                      | <ul style="list-style-type: none"><li>• Input: Text</li><li>• Output: Audio</li></ul>                                                                                                                                                                                              |
| Token limits                                                                                                                     | <ul style="list-style-type: none"><li>• Input: 8,192</li><li>• Output: 16,384</li></ul>                                                                                                                                                                                            |
| Speaker number support                                                                                                           | Single, multi-speaker (dialogue)                                                                                                                                                                                                                                                   |
| Supported output audio formats                                                                                                   | <ul style="list-style-type: none"><li>• Unary: LINEAR16 (default), ALAW, MULAW, MP3, OGG_OPUS, PCM</li><li>• Streaming: PCM (default), ALAW, MULAW, OGG_OPUS</li></ul>                                                                                                             |
| Region support                                                                                                                   | See <a href="#">Available regions</a> (#available_regions).                                                                                                                                                                                                                        |
| Voice options                                                                                                                    | See <a href="#">Voice options</a> (#voice_options).                                                                                                                                                                                                                                |
| Voice options                                                                                                                    | See <a href="#">Voice options</a> (#voice_options). Note that all voices for Gemini 3.1 Flash TTS are in <a href="#">Preview</a> ( <a href="https://cloud.google.com/products#product-launch-stages">https://cloud.google.com/products#product-launch-stages</a> ).                |
| Consumption options                                                                                                              | See <a href="#">Standard Paygo</a> (/vertex-ai/generative-ai/docs/standard-paygo).                                                                                                                                                                                                 |
| Example                                                                                                                          | <pre>model: "gemini-3.1-flash-tts-preview" prompt: "You are having a casual conversation with a friend.         Say the following in a friendly and amused way." text: "[laughs] I did NOT expect that. [sigh] Can you believe it!" speaker: "Callirrhoe"</pre> <p>0:00 / 0:05</p> |

# Additional controls

Additional controls and capabilities include the following:

- 1. **Natural conversation:** Voice interactions of remarkable quality, more appropriate expressivity, and patterns of rhythm are delivered with very low latency so you can converse fluidly.
- 2. **Style control:** Using natural language prompts, you can adapt the delivery within the conversation by steering it to adopt specific accents and produce a range of tones and expressions including a whisper.
- 3. **Dynamic performance:** These models can bring text to life for expressive readings of poetry, newscasts, and engaging storytelling. They can also perform with specific emotions and produce accents when requested.
- 4. **Enhanced pace and pronunciation control:** Controlling delivery speed helps to ensure more accuracy in pronunciation including specific words.

For information on how to choose the right API to use these voices in your application, see [Choose the right API](#) (#choose-the-right-api).

## Voice options

Gemini-TTS offers a wide range of voice options similar to our existing Chirp 3: HD Voices, each with distinct characteristics:

Select voice type ▾

| Name       | Gender | Demo        |
|------------|--------|-------------|
| Achernar   | Female | 0:00 / 0:10 |
| Achird     | Male   | 0:00 / 0:10 |
| Algenib    | Male   | 0:00 / 0:11 |
| Algieba    | Male   | 0:00 / 0:11 |
| Anilam     | Male   | 0:00 / 0:10 |
| Aoede      | Female | 0:00 / 0:12 |
| Autonoe    | Female | 0:00 / 0:12 |
| Callirrhoe | Female | 0:00 / 0:11 |
| Charon     | Male   | 0:00 / 0:11 |

| Name        | Gender | Demo        |
|-------------|--------|-------------|
| Despina     | Female | 0:00 / 0:11 |
| Enceladus   | Male   | 0:00 / 0:11 |
| Erinome     | Female | 0:00 / 0:11 |
| Fenrir      | Male   | 0:00 / 0:11 |
| Gacrux      | Female | 0:00 / 0:11 |
| Iapetus     | Male   | 0:00 / 0:11 |
| Kore        | Female | 0:00 / 0:11 |
| Laomedeia   | Female | 0:00 / 0:11 |
| Leda        | Female | 0:00 / 0:12 |
| Orus        | Male   | 0:00 / 0:10 |
| Pulcherrima | Female | 0:00 / 0:10 |
| Puck        | Male   | 0:00 / 0:11 |
| Rasalgethi  | Male   | 0:00 / 0:11 |
| Sadachbia   | Male   | 0:00 / 0:10 |
| Sadaltager  | Male   | 0:00 / 0:11 |
| Schedar     | Male   | 0:00 / 0:10 |
| Sulafat     | Female | 0:00 / 0:13 |

| Name          | Gender | Demo        |
|---------------|--------|-------------|
| Umbriel       | Male   | 0:00 / 0:11 |
| Vindemiatrix  | Female | 0:00 / 0:11 |
| Zephyr        | Female | 0:00 / 0:11 |
| Zubenelgenubi | Male   | 0:00 / 0:12 |

## Available languages

Gemini-TTS supports the following languages:

Select launch stage ▾

Search by language or BCP-47 code

| Language                | BCP-47 code | Launch readiness |
|-------------------------|-------------|------------------|
| Arabic (Egypt)          | ar-EG       | GA               |
| Bangla (Bangladesh)     | bn-BD       | GA               |
| Dutch (Netherlands)     | nl-NL       | GA               |
| English (India)         | en-IN       | GA               |
| English (United States) | en-US       | GA               |
| French (France)         | fr-FR       | GA               |
| German (Germany)        | de-DE       | GA               |
| Hindi (India)           | hi-IN       | GA               |
| Indonesian (Indonesia)  | id-ID       | GA               |
| Italian (Italy)         | it-IT       | GA               |
| Japanese (Japan)        | ja-JP       | GA               |
| Korean (South Korea)    | ko-KR       | GA               |
| Marathi (India)         | mr-IN       | GA               |
| Polish (Poland)         | pl-PL       | GA               |
| Portuguese (Brazil)     | pt-BR       | GA               |
| Romanian (Romania)      | ro-RO       | GA               |
| Russian (Russia)        | ru-RU       | GA               |
| Spanish (Spain)         | es-ES       | GA               |
| Tamil (India)           | ta-IN       | GA               |
| Telugu (India)          | te-IN       | GA               |

| Language                   | BCP-47 code | Launch readiness |
|----------------------------|-------------|------------------|
| Thai (Thailand)            | th-TH       | GA               |
| Turkish (Turkey)           | tr-TR       | GA               |
| Ukrainian (Ukraine)        | uk-UA       | GA               |
| Vietnamese (Vietnam)       | vi-VN       | GA               |
| Afrikaans (South Africa)   | af-ZA       | Preview          |
| Albanian (Albania)         | sq-AL       | Preview          |
| Amharic (Ethiopia)         | am-ET       | Preview          |
| Arabic (World)             | ar-001      | Preview          |
| Armenian (Armenia)         | hy-AM       | Preview          |
| Azerbaijani (Azerbaijan)   | az-AZ       | Preview          |
| Basque (Spain)             | eu-ES       | Preview          |
| Belarusian (Belarus)       | be-BY       | Preview          |
| Bulgarian (Bulgaria)       | bg-BG       | Preview          |
| Burmese (Myanmar)          | my-MM       | Preview          |
| Catalan (Spain)            | ca-ES       | Preview          |
| Cebuano (Philippines)      | ceb-PH      | Preview          |
| Chinese, Mandarin (China)  | cmn-CN      | Preview          |
| Chinese, Mandarin (Taiwan) | cmn-tw      | Preview          |
| Croatian (Croatia)         | hr-HR       | Preview          |
| Czech (Czech Republic)     | cs-CZ       | Preview          |
| Danish (Denmark)           | da-DK       | Preview          |
| English (Australia)        | en-AU       | Preview          |
| English (United Kingdom)   | en-GB       | Preview          |
| Estonian (Estonia)         | et-EE       | Preview          |
| Filipino (Philippines)     | fil-PH      | Preview          |
| Finnish (Finland)          | fi-FI       | Preview          |
| French (Canada)            | fr-CA       | Preview          |
| Galician (Spain)           | gl-ES       | Preview          |
| Georgian (Georgia)         | ka-GE       | Preview          |
| Greek (Greece)             | el-GR       | Preview          |
| Gujarati (India)           | gu-IN       | Preview          |
| Haitian Creole (Haiti)     | ht-HT       | Preview          |
| Hebrew (Israel)            | he-IL       | Preview          |
| Hungarian (Hungary)        | hu-HU       | Preview          |
| Icelandic (Iceland)        | is-IS       | Preview          |
| Javanese (Java)            | jav-JV      | Preview          |

| Language                     | BCP-47 code | Launch readiness |
|------------------------------|-------------|------------------|
| Kannada (India)              | kn-IN       | Preview          |
| Konkani (India)              | kok-IN      | Preview          |
| Lao (Laos)                   | lo-LA       | Preview          |
| Latin (Vatican City)         | la-VA       | Preview          |
| Latvian (Latvia)             | lv-LV       | Preview          |
| Lithuanian (Lithuania)       | lt-LT       | Preview          |
| Luxembourgish (Luxembourg)   | lb-LU       | Preview          |
| Macedonian (North Macedonia) | mk-MK       | Preview          |
| Maithili (India)             | mai-IN      | Preview          |
| Malagasy (Madagascar)        | mg-MG       | Preview          |
| Malay (Malaysia)             | ms-MY       | Preview          |
| Malayalam (India)            | ml-IN       | Preview          |
| Mongolian (Mongolia)         | mn-MN       | Preview          |
| Nepali (Nepal)               | ne-NP       | Preview          |
| Norwegian, Bokmål (Norway)   | nb-NO       | Preview          |
| Norwegian, Nynorsk (Norway)  | nn-NO       | Preview          |
| Odia (India)                 | or-IN       | Preview          |
| Pashto (Afghanistan)         | ps-AF       | Preview          |
| Persian (Iran)               | fa-IR       | Preview          |
| Portuguese (Portugal)        | pt-PT       | Preview          |
| Punjabi (India)              | pa-IN       | Preview          |
| Serbian (Serbia)             | sr-RS       | Preview          |
| Sindhi (India)               | sd-IN       | Preview          |
| Sinhala (Sri Lanka)          | si-LK       | Preview          |
| Slovak (Slovakia)            | sk-SK       | Preview          |
| Slovenian (Slovenia)         | sl-SI       | Preview          |
| Spanish (Latin America)      | es-419      | Preview          |
| Spanish (Mexico)             | es-MX       | Preview          |
| Swahili (Kenya)              | sw-KE       | Preview          |
| Swedish (Sweden)             | sv-SE       | Preview          |
| Urdu (Pakistan)              | ur-PK       | Preview          |

## Available regions

Gemini-TTS is available multiple regions, through Cloud Text-to-Speech API or Vertex AI API.

The ML processing for these models occurs within the specific region or multi-region where the request is made. For more information , see [Data residency](https://cloud.google.com/vertex-ai/generative-ai/docs/learn/data-residency) (/vertex-ai/generative-ai/docs/learn/data-residency).

For Cloud Text-to-Speech API, the following regions are supported:

| Region                  | Country or Jurisdiction | Available models                                                                                                |
|-------------------------|-------------------------|-----------------------------------------------------------------------------------------------------------------|
| global                  | Global (Non-DRZ)        | gemini-3.1-flash-tts-preview<br>gemini-2.5-flash-tts<br>gemini-2.5-pro-tts<br>gemini-2.5-flash-lite-preview-tts |
| us                      | United States           | gemini-2.5-flash-tts<br>gemini-2.5-pro-tts<br>gemini-2.5-flash-lite-preview-tts                                 |
| eu                      | European Union          | gemini-2.5-flash-tts<br>gemini-2.5-pro-tts<br>gemini-2.5-flash-lite-preview-tts                                 |
| northamerica-northeast1 | Canada                  | gemini-2.5-flash-tts<br>gemini-2.5-flash-lite-preview-tts                                                       |

These regions can be accessed using the following API endpoints: <REGION>-texttospeech.googleapis.com. Note the global region doesn't have a prefix: texttospeech.googleapis.com.

For Vertex AI API, the following regions are supported:

| Region                  | Country or Jurisdiction | Available models                                                                                                |
|-------------------------|-------------------------|-----------------------------------------------------------------------------------------------------------------|
| global                  | Global (Non-DRZ)        | gemini-3.1-flash-tts-preview<br>gemini-2.5-flash-tts<br>gemini-2.5-pro-tts<br>gemini-2.5-flash-lite-preview-tts |
| europe-central2         | European Union          | gemini-2.5-flash-tts<br>gemini-2.5-pro-tts<br>gemini-2.5-flash-lite-preview-tts                                 |
| europe-north1           | European Union          | gemini-2.5-flash-tts<br>gemini-2.5-pro-tts<br>gemini-2.5-flash-lite-preview-tts                                 |
| europe-southwest1       | European Union          | gemini-2.5-flash-tts<br>gemini-2.5-pro-tts<br>gemini-2.5-flash-lite-preview-tts                                 |
| europe-west1            | European Union          | gemini-2.5-flash-tts<br>gemini-2.5-pro-tts<br>gemini-2.5-flash-lite-preview-tts                                 |
| europe-west4            | European Union          | gemini-2.5-flash-tts<br>gemini-2.5-pro-tts<br>gemini-2.5-flash-lite-preview-tts                                 |
| northamerica-northeast1 | Canada                  | gemini-2.5-flash-tts<br>gemini-2.5-flash-lite-preview-tts                                                       |
| us-central1             | United States           | gemini-2.5-flash-tts<br>gemini-2.5-pro-tts<br>gemini-2.5-flash-lite-preview-tts                                 |
| us-east1                | United States           | gemini-2.5-flash-tts<br>gemini-2.5-pro-tts<br>gemini-2.5-flash-lite-preview-tts                                 |
| us-east4                | United States           | gemini-2.5-flash-tts<br>gemini-2.5-pro-tts<br>gemini-2.5-flash-lite-preview-tts                                 |

| Region    | Country or Jurisdiction | Available models                                                                |
|-----------|-------------------------|---------------------------------------------------------------------------------|
| us-east5  | United States           | gemini-2.5-flash-tts<br>gemini-2.5-pro-tts<br>gemini-2.5-flash-lite-preview-tts |
| us-south1 | United States           | gemini-2.5-flash-tts<br>gemini-2.5-pro-tts<br>gemini-2.5-flash-lite-preview-tts |
| us-west1  | United States           | gemini-2.5-flash-tts<br>gemini-2.5-pro-tts<br>gemini-2.5-flash-lite-preview-tts |
| us-west4  | United States           | gemini-2.5-flash-tts<br>gemini-2.5-pro-tts<br>gemini-2.5-flash-lite-preview-tts |

These regions can be accessed using the following API endpoints: `<REGION>-aiplatform.googleapis.com`. Note the `global` region doesn't have a prefix: `aiplatform.googleapis.com`.

## Choose the right API

Discover how to use Gemini-TTS models to synthesize single-speaker and multi-speaker speech using Cloud Text-to-Speech API or [Vertex AI API](#) (/vertex-ai/generative-ai/docs/sdks/overview).

Gemini-TTS is available through two APIs to help simplify the integration process for clients. Cloud Text-to-Speech API and Vertex AI API, with REST endpoints and SDKs in multiple programming languages.

Here are some guidelines to help you choose the right API.

If the following applies, choose Cloud Text-to-Speech API:

1. If you are using Chirp 3 HD or other voices, then you can continue to use the same Cloud Text-to-Speech API with minimal incremental updates.
2. If you need specific output encoding types, Cloud Text-to-Speech API can specify audio encoding. In Vertex AI API, the output is PCM 16bit 24k audio data, and doesn't have WAV headers. If you want to convert to another audio format, then the conversion needs to be handled from the client side.
3. If you must stream text in multiple chunks, Cloud Text-to-Speech API supports multiple requests and multiple responses interaction. In contrast, Vertex AI API supports single request and multiple responses.
4. If you want to apply [text normalization](#) (/python/docs/reference/texttospeech/latest/google.cloud.texttospeech\_v1.types.AdvancedVoiceOptions) by default. This is particularly useful for transcripts that include date and monetary values.

If the following applies, choose Vertex AI API:

1. If you are already using Gemini-TTS from AI Studio, you can switch to Vertex AI seamlessly to take advantage of the scalability and compliance of Google Cloud.
2. If you are using Vertex AI API for other models, then the unified api structure makes it easier to start using Gemini-TTS by specifying the model name and voice selection options.

## Use Cloud Text-to-Speech API

**Note:** The size of the text field and the prompt field individually can be at most 4,000 bytes. While the total size of the prompt and text fields can be up to 8,000 bytes, each field must be a maximum of 4000 bytes. The output audio can at most be around 655 seconds. If the input text amounts to audio longer than this limit, the audio will be truncated.

Optionally specify audio formats and sampling rates in "audioConfig" field if non-default options are needed.

| Description                   | Limit                                                                                                            | Type   |
|-------------------------------|------------------------------------------------------------------------------------------------------------------|--------|
| Text field                    | Less than or equal to 4,000 bytes.                                                                               | Input  |
| Prompt field                  | Less than or equal to 4,000 bytes.                                                                               | Input  |
| Text and prompt fields        | Less than or equal to 8,000 bytes.                                                                               | Input  |
| Duration for the output audio | Approximately 655 seconds. If the input text results in the audio exceeding 655 seconds, the audio is truncated. | Output |

**Note:** To be able to use Gemini-TTS, `aiplatform.endpoints.predict` permission is required for the model endpoint. This permission can be granted with the [roles/aiplatform.user role](/vertex-ai/docs/general/access-control#aiplatform.user) (/vertex-ai/docs/general/access-control#aiplatform.user).

## Before you begin

Before you can begin using Cloud Text-to-Speech, you must enable the API in the Google Cloud console by following steps:

1. Enable Cloud Text-to-Speech on a project.
2. Make sure billing is enabled for Cloud Text-to-Speech.
3. Set up authentication for your development environment.
4. Assign `aiplatform.endpoints.predict` permission for the authenticated user. This permission can be granted with the [roles/aiplatform.user role](/vertex-ai/docs/general/access-control#aiplatform.user) (/vertex-ai/docs/general/access-control#aiplatform.user).
5. Select the correct API endpoint based on [Available regions](#) (#available\_regions).

### Set up your Google Cloud project

1. Sign in to Google Cloud console (<https://console.cloud.google.com/>)
2. Go to the project selector page (<https://console.cloud.google.com/projectselector2/home/dashboard>)

You can either choose an existing project or create a new one. For more details about creating a project, see the [Google Cloud documentation](#) (/resource-manager/docs/creating-managing-projects).

3. If you create a new project, a message appears informing you to link a billing account. If you are using a pre-existing project, make sure to enable billing

[Learn how to confirm that billing is enabled for your project](#) (/billing/docs/how-to/modify-project)

- ★ **Note:** You must enable billing to use Cloud Text-to-Speech API, however, you won't be charged unless you exceed the free quota. For more information about pricing, see the [pricing](https://cloud.google.com/text-to-speech/pricing) (<https://cloud.google.com/text-to-speech/pricing>) page.
4. After you've selected a project and linked it to a billing account, you can enable the Cloud Text-to-Speech API. Go to the **Search products and resources** bar at the top of the page, and type in "speech". Select the **Cloud Text-to-Speech API** from the list of results.
  5. To try Cloud Text-to-Speech without linking it to your project, choose the **Try this API** option. To enable the Cloud Text-to-Speech API for use with your project, click **Enable**.
  6. Set up authentication for your development environment. For instructions, see [Set up authentication for Cloud Text-to-Speech](#) (/text-to-speech/docs/authentication#authn-how-to).

## Setting API endpoint for preferred region

You can pick an endpoint based on [Available regions](#) (#available\_regions).

PythonCURL (#curl)  
(#python)

```
import os
from google.cloud import texttospeech

PROJECT_ID = os.getenv("GOOGLE_CLOUD_PROJECT")
TTS_LOCATION = os.getenv("GOOGLE_CLOUD_REGION")

API_ENDPOINT = (
    f"{TTS_LOCATION}-texttospeech.googleapis.com"
    if TTS_LOCATION != "global"
    else "texttospeech.googleapis.com"
)
client = texttospeech.TextToSpeechClient(https://docs.cloud.google.com/python/docs/reference/texttospeech/latest/google.cloud.texttospe
    client_options=ClientOptions(api_endpoint=API_ENDPOINT)
)
```

## Perform synchronous single-speaker synthesis

PythonCURL (#curl)  
(#python)

```
# google-cloud-texttospeech minimum version 2.29.0 is required.

import os
from google.cloud import texttospeech

PROJECT_ID = os.getenv("GOOGLE_CLOUD_PROJECT")

def synthesize(prompt: str, text: str, output_filepath: str = "output.mp3"):
    """Synthesizes speech from the input text and saves it to an MP3 file.

    Args:
        prompt: Styling instructions on how to synthesize the content in
            the text field.
        text: The text to synthesize.
        output_filepath: The path to save the generated audio file.
            Defaults to "output.mp3".
    """
    client = texttospeech.TextToSpeechClient(https://docs.cloud.google.com/python/docs/reference/texttospeech/latest/google.cloud.text

    synthesis_input = texttospeech.SynthesisInput(https://docs.cloud.google.com/python/docs/reference/texttospeech/latest/google.cloi

    # Select the voice you want to use.
    voice = texttospeech.VoiceSelectionParams(https://docs.cloud.google.com/python/docs/reference/texttospeech/latest/google.cloud.tex
        language_code="en-US",
        name="Charon", # Example voice, adjust as needed
        model_name="gemini-3.1-flash-tts-preview"
    )

    audio_config = texttospeech.AudioConfig(https://docs.cloud.google.com/python/docs/reference/texttospeech/latest/google.cloud.textt
        audio_encoding=texttospeech.AudioEncoding(https://docs.cloud.google.com/python/docs/reference/texttospeech/latest/google.cloi
    )

    # Perform the text-to-speech request on the text input with the selected
    # voice parameters and audio file type.
    response = client.synthesize_speech(https://docs.cloud.google.com/python/docs/reference/texttospeech/latest/google.cloud.texttospee
        input=synthesis_input, voice=voice, audio_config=audio_config
    )

    # The response's audio_content is binary.
    with open(output_filepath, "wb") as out:
```

```
out.write(response.audio_content)
print(f"Audio content written to file: {output_filepath}")
```

## Perform streaming single-speaker synthesis

Streaming synthesis is suitable for real-time applications where fast response is critical for the user experience. In streaming connection, the API returns audio as it becomes available in small chunks.

As the caller of the API, make sure to consume the audio chunks, passing them down to your clients as they arrive (for example, using `socketio` for web apps).

Cloud Text-to-Speech API supports `multiple request multiple response` type streaming. While it's possible to send the input chunks to the API asynchronously as shown in the `request_generator`, the API only starts synthesizing when the client sends `Half-Close` as a signal that it won't send any more data to the API.

The `prompt` field must be set in the first input chunk, because it's ignored in consecutive chunks.

**Python**  
(#python)

```
# google-cloud-texttospeech minimum version 2.29.0 is required.

import datetime
import os
import numpy as np
from google.cloud import texttospeech

PROJECT_ID = os.getenv("GOOGLE_CLOUD_PROJECT")

def synthesize(prompt: str, text_chunks: list[str], model: str, voice: str, locale: str):
    """Synthesizes speech from the input text.

    Args:
        prompt: Styling instructions on how to synthesize the content in
            the text field.
        text_chunks: Text chunks to synthesize. Note that The synthesis will
            start when the client initiates half-close.
        model: gemini tts model name. gemini-3.1-flash-tts-preview, gemini-2.5-flash-tts, gemini-2.5-flash-lite-pr
        voice: voice name. Example: leda, kore. Refer to available voices.
        locale: locale name. Example: en-us. Refer to available locales.
    """
    client = texttospeech.TextToSpeechClient(https://docs.cloud.google.com/python/docs/reference/texttospeech/latest/google.cloud.texttospeech.TextToSpeechClient)

    config_request = texttospeech.StreamingSynthesizeRequest(https://docs.cloud.google.com/python/docs/reference/texttospeech/latest/google.cloud.texttospeech.StreamingSynthesizeRequest)
    streaming_config = texttospeech.StreamingSynthesizeConfig(https://docs.cloud.google.com/python/docs/reference/texttospeech/latest/google.cloud.texttospeech.StreamingSynthesizeConfig)
    voice = texttospeech.VoiceSelectionParams(https://docs.cloud.google.com/python/docs/reference/texttospeech/latest/google.cloud.texttospeech.VoiceSelectionParams)
    name = voice,
    language_code = locale,
    model_name = model
    )
    )
    )

# Example request generator. A function like this can be linked to an LLM
# text generator and the text can be passed to the TTS API asynchronously.
def request_generator():
    yield config_request

    for i, text in enumerate(text_chunks):
        yield texttospeech.StreamingSynthesizeRequest(https://docs.cloud.google.com/python/docs/reference/texttospeech/latest/google.cloud.texttospeech.StreamingSynthesizeRequest)
        input = texttospeech.StreamingSynthesisInput(https://docs.cloud.google.com/python/docs/reference/texttospeech/latest/google.cloud.texttospeech.StreamingSynthesisInput)
        text = text,
        # Prompt is only supported in the first input chunk.
```

```

        prompt=prompt if i == 0 else None,
    )
)

request_start_time = datetime.datetime.now()
streaming_responses = client.streaming_synthesize (https://docs.cloud.google.com/python/docs/reference/texttospeech/latest/googl

is_first_chunk_received = False
final_audio_data = np.array([])
num_chunks_received = 0
for response in streaming_responses:
    # just a simple progress indicator
    num_chunks_received += 1
    print(".", end="")
    if num_chunks_received % 40 == 0:
        print("")

    # measuring time to first audio
    if not is_first_chunk_received:
        is_first_chunk_received = True
        first_chunk_received_time = datetime.datetime.now()

    # accumulating audio. In a web-server scenario, you would want to
    # "emit" audio to the frontend as soon as it arrives.
    #
    # For example using flask socketio, you could do the following
    # from flask_socketio import SocketIO, emit
    # emit("audio", response.audio_content)
    # socketio.sleep(0)
    audio_data = np.frombuffer(response.audio_content, dtype=np.int16)
    final_audio_data = np.concatenate((final_audio_data, audio_data))

time_to_first_audio = first_chunk_received_time - request_start_time
time_to_completion = datetime.datetime.now() - request_start_time
audio_duration = len(final_audio_data) / 24_000 # default sampling rate.

print("\n")
print(f"Time to first audio: {time_to_first_audio.total_seconds()} seconds")
print(f"Time to completion: {time_to_completion.total_seconds()} seconds")
print(f"Audio duration: {audio_duration} seconds")

return final_audio_data

```

## Perform synchronous multi-speaker synthesis with freeform text input

**Note:** The combined size of all lines of dialogue can be at most 4,000 bytes. While the total size of the prompt and dialogue can be up to 4,000 bytes, each prompt and dialogue field must be a maximum of 4,000 bytes. Speaker aliases must consist solely of alphanumeric characters, excluding whitespace.

| Description                   | Constraints & Limits                                                                                             | Type   |
|-------------------------------|------------------------------------------------------------------------------------------------------------------|--------|
| Text field                    | Less than or equal to 4,000 bytes                                                                                | Input  |
| Prompt field                  | Less than or equal to 4,000 bytes                                                                                | Input  |
| Text and prompt fields        | Less than or equal to 8,000 bytes                                                                                | Input  |
| Speaker aliases               | Alphanumeric characters and no whitespace                                                                        | Input  |
| Duration for the output audio | Approximately 655 seconds. If the input text results in the audio exceeding 655 seconds, the audio is truncated. | Output |

PythonCURL (#curl)  
(#python)

```

# google-cloud-texttospeech minimum version 2.31.0 is required.

import os
from google.cloud import texttospeech

PROJECT_ID = os.getenv("GOOGLE_CLOUD_PROJECT")

def synthesize_multispeaker_freeform(
    prompt: str,
    text: str,
    output_filepath: str = "output_non_turn_based.wav",
):
    """Synthesizes speech from non-turn-based input and saves it to a WAV file.

    Args:
        prompt: Styling instructions on how to synthesize the content in the
            text field.
        text: The text to synthesize, containing speaker aliases to indicate
            different speakers. Example: "Sam: Hi Bob!\nBob: Hi Sam!"
        output_filepath: The path to save the generated audio file. Defaults to
            "output_non_turn_based.wav".
    """
    client = texttospeech.TextToSpeechClient(https://docs.cloud.google.com/python/docs/reference/texttospeech/latest/google.cloud.textt
    synthesis_input = texttospeech.SynthesisInput(https://docs.cloud.google.com/python/docs/reference/texttospeech/latest/google.clou
    multi_speaker_voice_config = texttospeech.MultiSpeakerVoiceConfig(https://docs.cloud.google.com/python/docs/reference/textt
        speaker_voice_configs=[
            texttospeech.MultispeakerPrebuiltVoice(https://docs.cloud.google.com/python/docs/reference/texttospeech/latest/google.cl
                speaker_alias="Speaker1",
                speaker_id="Kore",
            ),
            texttospeech.MultispeakerPrebuiltVoice(https://docs.cloud.google.com/python/docs/reference/texttospeech/latest/google.cl
                speaker_alias="Speaker2",
                speaker_id="Charon",
            ),
        ]
    )

    voice = texttospeech.VoiceSelectionParams(https://docs.cloud.google.com/python/docs/reference/texttospeech/latest/google.cloud.tex
        language_code="en-US",
        model_name="gemini-3.1-flash-tts-preview",
        multi_speaker_voice_config=multi_speaker_voice_config,
    )

    audio_config = texttospeech.AudioConfig(https://docs.cloud.google.com/python/docs/reference/texttospeech/latest/google.cloud.textt
        audio_encoding=texttospeech.AudioEncoding(https://docs.cloud.google.com/python/docs/reference/texttospeech/latest/google.clou
        sample_rate_hertz=24000,
    )

    response = client.synthesize_speech(https://docs.cloud.google.com/python/docs/reference/texttospeech/latest/google.cloud.texttospee
        input=synthesis_input, voice=voice, audio_config=audio_config
    )

    with open(output_filepath, "wb") as out:
        out.write(response.audio_content)
        print(f"Audio content written to file: {output_filepath}")

```

## Perform synchronous multi-speaker synthesis with structured text input

Multi-speaker with structured text input enables intelligent verbalization of text in a human-like way. For example, this kind of input is useful for addresses and dates. Freeform text input speaks the text exactly as written.

**Note:** Speaker aliases must consist solely of alphanumeric characters, excluding whitespace. The combined size of all lines of dialogue can be at most 4,000 bytes. While the total size of the prompt and dialogue can be up to 4,000 bytes, each prompt and dialogue field must be a maximum of 4,000 bytes.

| Description                          | Constraints & Limits                                                                                             | Type   |
|--------------------------------------|------------------------------------------------------------------------------------------------------------------|--------|
| MultiSpeakerMarkUp field             | Less than or equal to 4,000 bytes                                                                                | Input  |
| Prompt field                         | Less than or equal to 4,000 bytes                                                                                | Input  |
| MultiSpeakerMarkUp and prompt fields | Less than or equal to 8,000 bytes                                                                                | Input  |
| Speaker aliases                      | Alphanumeric characters and no whitespace                                                                        | Input  |
| Duration for the output audio        | Approximately 655 seconds. If the input text results in the audio exceeding 655 seconds, the audio is truncated. | Output |

PythonCURL (#curl)

(#python)

```
# google-cloud-texttospeech minimum version 2.31.0 is required.

import os
from google.cloud import texttospeech

PROJECT_ID = os.getenv("GOOGLE_CLOUD_PROJECT")

def synthesize_multispeaker_structured(
    prompt: str,
    turns: list[texttospeech.MultiSpeakerMarkup (https://docs.cloud.google.com/python/docs/reference/texttospeech/latest/google.cloud.
    output_filepath: str = "output_turn_based.wav",
):
    """Synthesizes speech from turn-based input and saves it to a WAV file.

    Args:
        prompt: Styling instructions on how to synthesize the content in the
            text field.
        turns: A list of texttospeech.MultiSpeakerMarkup.Turn objects representing
            the dialogue turns.
        output_filepath: The path to save the generated audio file. Defaults to
            "output_turn_based.wav".
    """
    client = texttospeech.TextToSpeechClient (https://docs.cloud.google.com/python/docs/reference/texttospeech/latest/google.cloud.text

    synthesis_input = texttospeech.SynthesisInput (https://docs.cloud.google.com/python/docs/reference/texttospeech/latest/google.clou
        multi_speaker_markup=texttospeech.MultiSpeakerMarkup (https://docs.cloud.google.com/python/docs/reference/texttospeech/lat
        prompt=prompt,
    )

    multi_speaker_voice_config = texttospeech.MultiSpeakerVoiceConfig (https://docs.cloud.google.com/python/docs/reference/textt
        speaker_voice_configs=[
            texttospeech.MultispeakerPrebuiltVoice (https://docs.cloud.google.com/python/docs/reference/texttospeech/latest/google.cl
                speaker_alias="Speaker1",
                speaker_id="Kore",
            ),
            texttospeech.MultispeakerPrebuiltVoice (https://docs.cloud.google.com/python/docs/reference/texttospeech/latest/google.cl
                speaker_alias="Speaker2",
                speaker_id="Charon",
            ),
        ]
    )

    voice = texttospeech.VoiceSelectionParams (https://docs.cloud.google.com/python/docs/reference/texttospeech/latest/google.cloud.tex
        language_code="en-US",
        model_name="gemini-3.1-flash-tts-preview",
        multi_speaker_voice_config=multi_speaker_voice_config,
```

```

)

audio_config = texttospeech.AudioConfig (https://docs.cloud.google.com/python/docs/reference/texttospeech/latest/google.cloud.texttospeech.AudioConfig (https://docs.cloud.google.com/python/docs/reference/texttospeech/latest/google.cloud.texttospeech.AudioEncoding (https://docs.cloud.google.com/python/docs/reference/texttospeech/latest/google.cloud.texttospeech.AudioEncoding
sample_rate_hertz=24000,
)

response = client.synthesize_speech (https://docs.cloud.google.com/python/docs/reference/texttospeech/latest/google.cloud.texttospeech.synthesize_speech
input=synthesis_input, voice=voice, audio_config=audio_config
)

with open(output_filepath, "wb") as out:
    out.write(response.audio_content)
    print(f"Audio content written to file: {output_filepath}")

```

## Use Vertex AI API

Discover how to use Gemini-TTS models to synthesize single-speaker and multi-speaker speech using Vertex AI API.

In Cloud Text-to-Speech API, the user can provide text and prompt fields separately. In Vertex AI API, this is a single `contents` field, where it specifies the input in the form of "{prompt}: {text}", eg. "Say the following in a curious way: OK, so... tell me about this AI thing."

**Note:** The total size of the prompt and text fields can be up to 8,000 bytes. The output audio can at most be around 655 seconds. If the input text amounts to audio longer than this limit, the audio is truncated.

| Description                   | Limit                                                                                                            | Type   |
|-------------------------------|------------------------------------------------------------------------------------------------------------------|--------|
| Contents field                | Less than or equal to 8,000 bytes.                                                                               | Input  |
| Duration for the output audio | Approximately 655 seconds. If the input text results in the audio exceeding 655 seconds, the audio is truncated. | Output |

## Before you begin

Before you can begin, follow the installation steps for [Vertex AI API](#) (/vertex-ai/generative-ai/docs/sdks/overview). To take advantage of data residency options, make sure to set the `LOCATION` based on [Available regions](#) (#available\_regions).

**Note:** The API ignores the `temperature`, `top_k` and `top_p` generation config parameters in the request.

## Perform synchronous single-speaker synthesis

These code samples demonstrate how to perform synchronous single-speaker synthesis.

**Python**  
(#python)

```

from google import genai
from google.genai import types
import wave
import os

PROJECT_ID = os.getenv("GOOGLE_CLOUD_PROJECT")
LOCATION = os.environ.get("GOOGLE_CLOUD_REGION", "global")

# Set up the wave file to save the output:
def wave_file(filename, pcm, channels=1, rate=24000, sample_width=2):
    with wave.open(filename, "wb") as wf:

```

```

        wf.setnchannels(channels)
        wf.setsampwidth(sample_width)
        wf.setframerate(rate)
        wf.writeframes(pcm)

client = genai.Client(vertexai=True, project=PROJECT_ID, location=LOCATION)

response = client.models.generate_content(
    model="gemini-3.1-flash-tts-preview",
    contents="Say the following in a curious way: OK, so... tell me about this [uhm] AI thing.",
    config=types.GenerateContentConfig(
        speech_config=types.SpeechConfig(
            language_code="en-in",
            voice_config=types.VoiceConfig(
                prebuilt_voice_config=types.PrebuiltVoiceConfig(
                    voice_name='Kore',
                )
            )
        ),
    ),
)

data = response.candidates[0].content.parts[0].inline_data.data

file_name='output_speech.wav'
wave_file(file_name, data) # Saves the file to current directory

```

## Perform streaming single-speaker synthesis

In Vertex AI API, unidirectional streaming is supported, that is, the client sends a single request and receives a stream of responses.

This code sample demonstrates how to perform streaming single-speaker synthesis.

Python  
(#python)

```

from google import genai
from google.genai import types
import wave
import os
import datetime

PROJECT_ID = os.getenv("GOOGLE_CLOUD_PROJECT")
LOCATION = os.environ.get("GOOGLE_CLOUD_REGION", "global")

# Set up the wave file to save the output:
def wave_file(filename, pcm, channels=1, rate=24000, sample_width=2):
    with wave.open(filename, "wb") as wf:
        wf.setnchannels(channels)
        wf.setsampwidth(sample_width)
        wf.setframerate(rate)
        wf.writeframes(pcm)

def synthesize(text: str, model: str, voice: str, locale: str):
    """Synthesizes speech from the input text.

    Args:
        text: Text to synthesize.
        model: gemini tts model name. Available models are gemini-3.1-flash-tts-preview, gemini-2.5-flash-tts, gem:
        voice: voice name. Example: leda, kore. Refer to available voices.
        locale: locale name. Example: en-us. Refer to available locales.
    """
    client = genai.Client(vertexai=True, project=PROJECT_ID, location=LOCATION)

```

```

generate_content_config=types.GenerateContentConfig(
    speech_config=types.SpeechConfig(
        language_code=locale,
        voice_config=types.VoiceConfig(
            prebuilt_voice_config=types.PrebuiltVoiceConfig(
                voice_name=voice,
            )
        )
    ),
)

request_start_time = datetime.datetime.now()
is_first_chunk_received = False
final_audio_data = bytes()
num_chunks_received = 0

for chunk in client.models.generate_content_stream(
    model=model,
    contents=text,
    config=generate_content_config,
):
    # just a simple progress indicator
    num_chunks_received += 1
    print(".", end="")
    if num_chunks_received % 40 == 0:
        print("")
    # measuring time to first audio
    if not is_first_chunk_received:
        is_first_chunk_received = True
        first_chunk_received_time = datetime.datetime.now()

    if (
        chunk.candidates is None
        or not chunk.candidates
        or chunk.candidates[0].content is None
        or not chunk.candidates[0].content.parts
    ):
        continue
    part = chunk.candidates[0].content.parts[0]
    if part.inline_data and part.inline_data.data:
        # accumulating audio. In a web-server scenario, you would want to
        # "emit" audio to the frontend as soon as it arrives.
        #
        # For example using flask socketio, you could do the following
        # from flask_socketio import SocketIO, emit
        # emit("audio", chunk.candidates[0].content.parts[0].inline_data.data)
        # socketio.sleep(0)
        final_audio_data += chunk.candidates[0].content.parts[0].inline_data.data

time_to_first_audio = first_chunk_received_time - request_start_time
time_to_completion = datetime.datetime.now() - request_start_time

print("\n")
print(f"Time to first audio: {time_to_first_audio.total_seconds()} seconds")
print(f"Time to completion: {time_to_completion.total_seconds()} seconds")

return final_audio_data

audio_data = synthesize(
    "Say the following in a curious way: Radio Bakery is a New York City gem, celebrated for its exceptional and c
    "gemini-3.1-flash-tts-preview",
    "Kore",
    "en-in")
file_name='output_speech.wav'
wave_file(file_name, audio_data)

```

```
Audio("output_speech.wav")
```

## Perform synchronous multi-speaker synthesis

This code sample demonstrates how to perform synchronous multi-speaker synthesis.

**PythonCURL** (#curl)  
(#python)

```
from google import genai
from google.genai import types
import wave
import os
import datetime

PROJECT_ID = os.getenv("GOOGLE_CLOUD_PROJECT")
LOCATION = os.environ.get("GOOGLE_CLOUD_REGION", "global")

prompt = """TTS the following conversation between Joe and Jane:
    Joe: How's it going today Jane?
    Jane: Not too bad, how about you?"""

# Set up the wave file to save the output:
def wave_file(filename, pcm, channels=1, rate=24000, sample_width=2):
    with wave.open(filename, "wb") as wf:
        wf.setnchannels(channels)
        wf.setsampwidth(sample_width)
        wf.setframerate(rate)
        wf.writeframes(pcm)

client = genai.Client(vertexai=True, project=PROJECT_ID, location=LOCATION)

response = client.models.generate_content(
    model="gemini-3.1-flash-tts-preview",
    contents=prompt,
    config=types.GenerateContentConfig(
        speech_config=types.SpeechConfig(
            language_code="en-in",
            multi_speaker_voice_config=types.MultiSpeakerVoiceConfig(
                speaker_voice_configs=[
                    types.SpeakerVoiceConfig(
                        speaker='Joe',
                        voice_config=types.VoiceConfig(
                            prebuilt_voice_config=types.PrebuiltVoiceConfig(
                                voice_name='Kore',
                            )
                        )
                    ),
                    types.SpeakerVoiceConfig(
                        speaker='Jane',
                        voice_config=types.VoiceConfig(
                            prebuilt_voice_config=types.PrebuiltVoiceConfig(
                                voice_name='Puck',
                            )
                        )
                    )
                ]
            )
        )
    )

data = response.candidates[0].content.parts[0].inline_data.data
```

```

file_name='output_speech.wav'
wave_file(file_name, data) # Saves the file to current directory

Audio("output_speech.wav")

```

## Perform streaming multi-speaker synthesis

In Vertex AI API, unidirectional streaming is supported, that is, the client sends a single request and receives a stream of responses.

This code sample demonstrates how to perform streaming multi-speaker synthesis.

Python  
(#python)

```

from google import genai
from google.genai import types
import wave
import os
import datetime

PROJECT_ID = os.getenv("GOOGLE_CLOUD_PROJECT")
LOCATION = os.environ.get("GOOGLE_CLOUD_REGION", "global")

prompt = """TTS the following conversation between Joe and Jane:
    Joe: How's it going today Jane?
    Jane: Not too bad! Honestly, I'm just blown away by this new Gemini Text-to-Speech model.
    Joe: Right?! This is a total game changer! And do you know what I just discovered?
    Jane: What?
    Joe: You can use the multi-speaker configuration to create a conversation, just like us!
    Jane: Just like us!"""

# Set up the wave file to save the output:
def wave_file(filename, pcm, channels=1, rate=24000, sample_width=2):
    with wave.open(filename, "wb") as wf:
        wf.setnchannels(channels)
        wf.setsampwidth(sample_width)
        wf.setframerate(rate)
        wf.writeframes(pcm)

def synthesize(text: str, model: str, voice: str, locale: str):
    """Synthesizes speech from the input text.

    Args:
        text: Text to synthesize. Note that The synthesis will start when the
            client initiates half-close.
        model: gemini tts model name. Available models are: gemini-3.1-flash-tts-preview, gemini-2.5-flash-tts, gei
        voice: voice name. Example: leda, kore. Refer to available voices.
        locale: locale name. Example: en-us. Refer to available locales.
    """
    client = genai.Client(vertexai=True, project=PROJECT_ID, location=LOCATION)

    generate_content_config=types.GenerateContentConfig(
        speech_config=types.SpeechConfig(
            language_code="en-in",
            multi_speaker_voice_config=types.MultiSpeakerVoiceConfig(
                speaker_voice_configs=[
                    types.SpeakerVoiceConfig(
                        speaker='Joe',
                        voice_config=types.VoiceConfig(
                            prebuilt_voice_config=types.PrebuiltVoiceConfig(
                                voice_name='Kore',
                            )
                        )
                    )
                ]
            )
        )
    )

```

```

        ),
        types.SpeakerVoiceConfig(
            speaker='Jane',
            voice_config=types.VoiceConfig(
                prebuilt_voice_config=types.PrebuiltVoiceConfig(
                    voice_name='Puck',
                )
            )
        ),
    ],
),
),
)

request_start_time = datetime.datetime.now()
is_first_chunk_received = False
final_audio_data = bytes()
num_chunks_received = 0

for chunk in client.models.generate_content_stream(
    model=model,
    contents=text,
    config=generate_content_config,
):
    # just a simple progress indicator
    num_chunks_received += 1
    print(".", end="")
    if num_chunks_received % 40 == 0:
        print("")
    # measuring time to first audio
    if not is_first_chunk_received:
        is_first_chunk_received = True
        first_chunk_received_time = datetime.datetime.now()

    if (
        chunk.candidates is None
        or not chunk.candidates
        or chunk.candidates[0].content is None
        or not chunk.candidates[0].content.parts
    ):
        continue
    part = chunk.candidates[0].content.parts[0]
    if part.inline_data and part.inline_data.data:
        # accumulating audio. In a web-server scenario, you would want to
        # "emit" audio to the frontend as soon as it arrives.
        #
        # For example using flask socketio, you could do the following
        # from flask_socketio import SocketIO, emit
        # emit("audio", chunk.candidates[0].content.parts[0].inline_data.data)
        # socketio.sleep(0)
        final_audio_data += chunk.candidates[0].content.parts[0].inline_data.data

    time_to_first_audio = first_chunk_received_time - request_start_time
    time_to_completion = datetime.datetime.now() - request_start_time

    print("\n")
    print(f"Time to first audio: {time_to_first_audio.total_seconds()} seconds")
    print(f"Time to completion: {time_to_completion.total_seconds()} seconds")

    return final_audio_data

audio_data = synthesize(
    prompt,
    "gemini-3.1-flash-tts-preview",
    "Kore",
    "en-in")
file_name='output_speech.wav'
wave_file(file_name, audio_data)

```

```
Audio("output_speech.wav")
```

## Perform speech synthesis in Media Studio

### + Permissions required for this task

To perform this task, you must have the following [permissions](#) (/iam/docs/overview#permissions):

- `serviceusage.services.use`

You can use the Media Studio in the Google Cloud console to experiment with text-to-speech models. This provides a user interface for quickly generating, listening to synthesized audio and experimenting with different style instructions and parameters.

1. In the Google Cloud console, go to the **Vertex AI Studio > Media Studio** page.

**Media Studio** (<https://console.cloud.google.com/vertex-ai/studio/media/generate;tab=audio>)

2. Select **Speech** from the media drop-down.

3. In the text field, enter the text you want to synthesize into speech.

4. In the **Settings** pane, configure the following settings:

- a. **Model:** Select the Cloud TTS (TTS) model that you want to use, such as **Gemini 2.5 Pro TTS**. For more information about available models, see [Cloud TTS models](#) (/text-to-speech/docs/list-voices-and-types).
  - b. **Style instructions:** Optional: Enter a text prompt that describes the selected speaking style, tone, and emotional delivery. This lets you to guide the model's performance beyond the default narration. For example: *"Narrate in a calm, professional tone for a documentary."*
  - c. **Language:** Select the language and region of the input text. The model generates speech in the selected language and accent. For example, *English (United States)*.
  - d. **Voice:** Choose a predefined voice for the narration. The list contains the available voices for the selected model and language, such as *Acherner (Female)*.
5. Optional: Expand the Advanced options section to configure technical audio settings:
    - a. **Audio encoding:** Select the encoding for the output audio file. **LINEAR16** is a lossless, uncompressed format suitable for high-quality audio processing. **MULAW** is also available for compressed audio output.
    - b. **Audio sample rate:** Select the sample rate in hertz (Hz). This determines the audio quality. Higher values like 44,100 Hz represent higher fidelity audio, equivalent to CD quality.
    - c. **Speed:** Adjust the speaking rate by moving the slider or entering a value. Values less than 1 slow down the speech, and values greater than 1 speed it up. The default is 1.
    - d. **Volume gain (db):** Adjust the volume of the output audio in decibels (dB). Positive values increase the volume, and negative values decrease it. The default is 0.

6. Click the **send** icon at the right of the text-box to generate the audio.

7. The generated audio appears in the media player. Click the play button to listen to the output. You can continue to adjust the settings, and generate new versions as needed.

## Prompting tips

Creating engaging and natural-sounding audio from text requires understanding the nuances of spoken language and translating them into script form. The following tips will help you craft scripts that sound authentic and capture the chosen tone.

# The three levers of speech control

For the most predictable and nuanced results, ensure all three of the following components are consistent with your desired output.

**Style Prompt** The primary driver of the overall emotional tone and delivery. The prompt sets the context for the entire speech segment.

- Example: You are an AI assistant speaking in a friendly and helpful tone.
- Example: Narrate this in the calm, authoritative tone of a nature documentary narrator.

**Text Content** The semantic meaning of the words you are synthesizing. An evocative phrase that is emotionally consistent with the style prompt will produce much more reliable results than neutral text.

- Good: A prompt for a scared tone works best with text like I think someone is in the house.
- Less Effective: A prompt for a scared tone with text like The meeting is at 4 PM. will produce ambiguous results.

**Markup Tags (Preview)** Bracketed tags like [sigh] are best used for injecting a specific, localized action or style modification, not for setting the overall tone. They work in concert with the style prompt and text content.

## Markup tag guide

### Preview

This product or feature is subject to the "Pre-GA Offerings Terms" in the General Service Terms section of the [Service Specific Terms \(/terms/service-terms#1\)](/terms/service-terms#1). Pre-GA products and features are available "as is" and might have limited support. For more information, see the [launch stage descriptions](https://cloud.google.com/products/#product-launch-stages) (<https://cloud.google.com/products/#product-launch-stages>).

Our research shows that bracketed markup tags operate in one of three distinct modes. Understanding a tag's mode is key to using it effectively.

### Mode 1: Non-speech sounds

The markup is replaced by an audible, non-speech vocalization (e.g., a sigh, a laugh). The tag itself is not spoken. These are excellent for adding realistic, human-like hesitations and reactions.

| Tag        | Behavior                    | Reliability | Guidance                                                                                                                                                     |
|------------|-----------------------------|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| [sigh]     | Inserts a sigh sound.       | High        | The emotional quality of the sigh is influenced by the prompt.                                                                                               |
| [laughing] | Inserts a laugh.            | High        | For best results, use a specific prompt. e.g., a generic prompt may yield a laugh of shock, while "react with an amused laugh" creates a laugh of amusement. |
| [uhm]      | Inserts a hesitation sound. | High        | Useful for creating a more natural, conversational feel.                                                                                                     |

### Mode 2: Style modifiers

The markup is not spoken, but it modifies the delivery of the subsequent speech. The scope and duration of the modification can vary.

| Tag        | Behavior                                           | Reliability | Guidance                                                                                                                                            |
|------------|----------------------------------------------------|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| [sarcasm]  | Imparts a sarcastic tone on the subsequent phrase. | High        | This tag is a powerful modifier. It demonstrates that abstract concepts can successfully steer the model's delivery.                                |
| [robotic]  | Makes the subsequent speech sound robotic.         | High        | The effect can extend across an entire phrase. A supportive style prompt (e.g., "Say this in a robotic way") is still recommended for best results. |
| [shouting] | Increases the volume of the subsequent speech.     | High        | Most effective when paired with a matching style prompt (e.g., "Shout this next part") and text that implies yelling.                               |

| Tag              | Behavior                                       | Reliability | Guidance                                                                                                                |
|------------------|------------------------------------------------|-------------|-------------------------------------------------------------------------------------------------------------------------|
| [whispering]     | Decreases the volume of the subsequent speech. | High        | Best results are achieved when the style prompt is also explicit (e.g., "now whisper this part as quietly as you can"). |
| [extremely fast] | Increases the speed of the subsequent speech.  | High        | Ideal for disclaimers or fast-paced dialogue. Minimal prompt support needed.                                            |

### Mode 3: Vocalized markup (adjectives)

The markup tag itself is spoken as a word, while also influencing the tone of the entire sentence. This behavior typically applies to emotional adjectives.

*Warning:* Because the tag itself is spoken, this mode is likely an undesired side effect for most use cases. Prefer using the Style Prompt to set these emotional tones instead.

| Tag       | Behavior                                                                        | Reliability | Guidance                                                                                                                                                                                      |
|-----------|---------------------------------------------------------------------------------|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| [scared]  | The word "scared" is spoken, and the sentence adopts a scared tone.             | High        | Performance is highly dependent on text content. The phrase "I just heard a window break" produces a genuinely scared result. A neutral phrase produces a "spooky" but less authentic result. |
| [curious] | The word "curious" is spoken, and the sentence adopts a curious tone.           | High        | Use an inquisitive phrase to support the tag's intent.                                                                                                                                        |
| [bored]   | The word "bored" is spoken, and the sentence adopts a bored, monotone delivery. | High        | Use with text that is mundane or repetitive for best effect.                                                                                                                                  |

### Mode 4: Pacing and pauses

These tags insert silence into the generated audio, giving you granular control over rhythm, timing, and pacing. Standard punctuation (commas, periods, semicolons) will also create natural pauses, but these tags offer more explicit control.

| Tag            | Behavior                                                        | Reliability | Guidance                                                                                                               |
|----------------|-----------------------------------------------------------------|-------------|------------------------------------------------------------------------------------------------------------------------|
| [short pause]  | Inserts a brief pause, similar to a comma (~250ms).             | High        | Use to separate clauses or list items for better clarity.                                                              |
| [medium pause] | Inserts a standard pause, similar to a sentence break (~500ms). | High        | Effective for separating distinct sentences or thoughts.                                                               |
| [long pause]   | Inserts a significant pause for dramatic effect (~1000ms+).     | High        | Use for dramatic timing. For example: "The answer is... [long pause] ...no." Avoid overuse, as it can sound unnatural. |

### Key strategies for reliable results

- **Align all three levers:** For maximum predictability, ensure your Style Prompt, Text Content, and any Markup Tags are all semantically consistent and working toward the same goal.
- **Use emotionally rich text:** Don't rely on prompts and tags alone. Give the model rich, descriptive text to work with. This is especially critical for nuanced emotions like sarcasm, fear, or excitement.
- **Write specific, detailed prompts:** The more specific your style prompt, the more reliable the result. "React with an amused laugh" is better than just [laughing]. "Speak like a 1940s radio news announcer" is better than "Speak in an old-fashioned way."
- **Test and verify new tags:** The behavior of a new or untested tag is not always predictable. A tag you assume is a style modifier might be vocalized. Always test a new tag or prompt combination to confirm its behavior before deploying to production.

## Safety check settings

While some safety checks cannot be disabled, you can adjust the thresholds for certain harm categories to further enhance safety and minimize misuse. This can be optionally enabled by setting safety filters under [AdvancedVoiceOptions](#) (/python/docs/reference/texttospeech/latest/google.cloud.texttospeech\_v1.types.AdvancedVoiceOptions).

There are [4 harm categories](#) (/vertex-ai/generative-ai/docs/multimodal/configure-safety-filters#harm\_categories) that can be configured at different levels of threshold. One or more categories can be set to block harmful content.

The legacy `relax_safety_filters` field is deprecated.

Depending on the safety filters that you configure, your output may contain a safety code similar to: "Cloud Text-to-Speech could not generate audio because the input text or prompt violates Vertex AI's usage guidelines. If you think this was an error, send feedback. Support codes: **62263041**"

The code listed in the output corresponds to a specific harmful category.

The following table displays the support code to safety category mappings:

| Support code | Safety category   | Description                     |
|--------------|-------------------|---------------------------------|
| 54702341     | Unspecified       | Unspecified category.           |
| 62263041     | Dangerous content | Potentially dangerous content.  |
| 57734940     | Harassment        | Harassment-related content.     |
| 90789179     | Sexually explicit | Sexual or suggestive content.   |
| 78610348     | Hate speech       | Content containing hate speech. |

These code examples demonstrate how to customize safety configurations in Cloud Text-to-Speech API. For usage in Vertex AI API, refer to [Vertex AI API examples](#) (/vertex-ai/generative-ai/docs/multimodal/configure-safety-filters#api-examples).

**Python**  
**CURL** (#curl)  
(#python)

```
# google-cloud-texttospeech minimum version 2.32.0 is required.
request = texttospeech.SynthesizeSpeechRequest(
    input=synthesis_input,
    voice=voice,
    audio_config=audio_config,
    advanced_voice_options=texttospeech.AdvancedVoiceOptions(
        safety_settings=texttospeech.AdvancedVoiceOptions.SafetySettings(
            settings=[texttospeech.AdvancedVoiceOptions.SafetySetting(
                category=texttospeech.AdvancedVoiceOptions.HarmCategory.HARM_CATEGORY_DANGEROUS_CONTENT,
                threshold=texttospeech.AdvancedVoiceOptions.HarmBlockThreshold.BLOCK_LOW_AND_ABOVE
            ),
                texttospeech.AdvancedVoiceOptions.SafetySetting(
                    category=texttospeech.AdvancedVoiceOptions.HarmCategory.HARM_CATEGORY_HARASSMENT,
                    threshold=texttospeech.AdvancedVoiceOptions.HarmBlockThreshold.BLOCK_LOW_AND_ABOVE
                ),
                texttospeech.AdvancedVoiceOptions.SafetySetting(
                    category=texttospeech.AdvancedVoiceOptions.HarmCategory.HARM_CATEGORY_HATE_SPEECH,
                    threshold=texttospeech.AdvancedVoiceOptions.HarmBlockThreshold.BLOCK_LOW_AND_ABOVE
                ),
                texttospeech.AdvancedVoiceOptions.SafetySetting(
                    category=texttospeech.AdvancedVoiceOptions.HarmCategory.HARM_CATEGORY_SEXUALLY_EXPLICIT,
                    threshold=texttospeech.AdvancedVoiceOptions.HarmBlockThreshold.BLOCK_LOW_AND_ABOVE
                )
            ]
        )
    ),
)
# Perform the text-to-speech request on the text input with the selected
# voice parameters and audio file type.
response = client.synthesize_speech(request=request)
```

Except as otherwise noted, the content of this page is licensed under the [Creative Commons Attribution 4.0 License](https://creativecommons.org/licenses/by/4.0/) (<https://creativecommons.org/licenses/by/4.0/>), and code samples are licensed under the [Apache 2.0 License](https://www.apache.org/licenses/LICENSE-2.0) (<https://www.apache.org/licenses/LICENSE-2.0>). For details, see the [Google Developers Site Policies](https://developers.google.com/site-policies) (<https://developers.google.com/site-policies>). Java is a registered trademark of Oracle and/or its affiliates.

Last updated 2026-08-26 UTC.